Meridian Sales Portal

Project Roadmap & Implementation Report

Prepared for engineering review | 21 July 2026

A full-stack, multi-role B2B commerce platform: client storefront & ordering, returns and refunds, real-time support chat, field-sales operations, and an admin control panel — built on Next.js, Express, and Supabase (PostgreSQL).

Contents

1. Executive Summary
2. Technology Stack & Architecture
3. Project Structure
4. Database Design
5. Implementation Roadmap — Phases 1–10
6. API Surface Reference
7. Security & Edge-Case Handling
8. Data-Quality Remediation: Duplicate Product Images
9. Performance Remediation: Slow Page Loads (this iteration)
10. UI/UX Snapshots
11. Verification & Test Results
12. Known Limitations & Future Roadmap

1. Executive Summary

The Meridian Sales Portal is a production-style wholesale commerce platform with three distinct user roles served from a single codebase: **Clients** (browse catalog, place orders, request returns, track refunds, chat with support), **Field sales agents** (visit scheduling, on-site order capture), and **Admins** (catalog management, order/return adjudication, user administration, support inbox).

The system comprises a Next.js (App Router) frontend, an Express.js REST backend hardened with Helmet, CORS allow-listing and layered rate limiting, and a Supabase-hosted PostgreSQL database with a fully normalized schema (products, categories, orders, returns, refunds, chat, field operations).

Metric	Value
Catalog size	797 products across categories (beauty, fragrances, furniture, groceries, electronics, footwear, apparel, ...)
Order history	1,568 seeded orders with realistic statuses and channels
Returns / refunds	46 returns, 30 refunds — full lifecycle (pending → approved → processing → completed)
Image uniqueness	797 products → 786 distinct primary images; 0 images shared across different base products
Integrity checks	12 / 12 automated data-integrity checks passing (see §8)

2. Technology Stack & Architecture

Layer	Technology	Notes
Frontend	Next.js (App Router), React, TypeScript, Tailwind CSS, SWR	Role-scoped route groups: /client, /field, /admin, /auth
Backend	Node.js + Express 5	REST API under /api/*, central error handler, health endpoint
Database	Supabase PostgreSQL (pooled)	SQL schema files + idempotent seed/migration scripts in backend/db/
Auth	Supabase Auth + JWT bearer tokens	Role claims enforced server-side per route
Security	Helmet, CORS origin allow-list, express-rate-limit	Auth routes: 50 req / 15 min; general API: 300 req / min; trust proxy = 1 hop
Images	DummyJSON CDN + curated Unsplash reserve	Live URL verification before any DB write

Request flow

Browser → Next.js rewrite proxy → Express API → Supabase PostgreSQL. The proxy hop is why `trust proxy` is set to exactly 1 — rate limiting keys off the real client IP without being spoofable via forged X-Forwarded-For chains.

```
frontend/ (Next.js) backend/ (Express)
  app/auth login, registration src/routes + src/controllers
  app/client catalog, orders,   auth, catalog, orders,
             returns, chat   returns-refunds, chat, field, admin
  app/field visits, field orders db/ schema.sql, portal-schema.sql,
  app/admin control panel   chat-schema.sql, field-schema.sql, seeds
```

3. Project Structure

Actual repository layout (directories verified against the live workspace):

```
meridian-sales-portal/
■■ frontend/                          Next.js App Router + TypeScript + Tailwind
■  ■■ app/
■  ■  ■■ page.tsx, layout.tsx, globals.css     landing + root shell
■  ■  ■■ auth/                            login / registration
■  ■  ■■ client/                          vendor portal
■  ■  ■  ■■ products/  launch/  inventory/    my products, launch wizard, stock
■  ■  ■  ■■ orders/  returns/  performance/  profile/
■  ■  ■■ field/                           field-officer app
■  ■  ■  ■■ sell/  leads/  orders/  products/  earnings/  performance/  join/  profile/
■  ■  ■■ admin/                           control panel
■  ■     ■■ products/  clients/  officers/  inbox/
■  ■■ components/   auth/ chat/ client/ landing/ launch/ portal/ ui/
■  ■■ lib/          api/ auth/ data/ store/ supabase/ validation/ types.ts utils.ts
■  ■■ hooks/  public/  middleware.ts
■  ■■ next.config.mjs                    /backend/* rewrite proxy to Express
■■ backend/                           Express 5 REST API
■  ■■ src/
■  ■  ■■ app.js                          helmet, CORS, rate limits, error handler
■  ■  ■■ config/                         supabase admin client
■  ■  ■■ middleware/auth.js             JWT auth + verification cache (§9)
■  ■  ■■ routes/                         auth, catalog, orders, returns-refunds, chat, fi
eld, admin
■  ■  ■■ controllers/                    matching controllers + refunds
■  ■■ db/
■     ■■ schema.sql  portal-schema.sql  chat-schema.sql  field-schema.sql
■     ■■ performance-indexes.sql          hot-path covering indexes (§9)
■     ■■ seed.js  seed-portal.js  seed-refunds.js  seed-client-returns.js
■     ■■ enrich-catalog.js  refresh-product-images.js
■     ■■ fix-duplicate-images.js          image de-duplication (§8)
■     ■■ migrate-*.js  create-admin.js  taxonomy.js  fix-return-refund-consistency.js
■■ PROJECT_ROADMAP.pdf                this document
■■ RETURNS_REFUNDS_IMPLEMENTATION.md   feature deep-dive
■■ REFUNDS_QUICKSTART.md              reviewer walkthrough
```

4. Database Design

Table	Purpose	Key constraints / notes
categories	Product taxonomy	Unique slug; FK from products
products	Catalog (name, sku, brand, price, mrp, stock, rating, images[], status)	Unique SKU; mrp ≥ price; text[] image gallery; status live/draft
orders	Client purchase records	amount = qty × unit_price enforced by seed + checks; status & channel enums
returns	Return requests	return_qty ≤ order qty; refund_amount ≤ order amount; reason codes; status workflow
refunds	Payment refunds	FK to returns/orders/clients; refund amount ≤ return amount; processed_at required when completed
chat threads / messages	Client ↔ support messaging	Thread ownership scoping; updated_at triggers

Table	Purpose	Key constraints / notes
field visits / field orders	Field-agent operations	Agent-scoped; visit scheduling and outcomes

All tables carry NOT NULL constraints on required fields, CHECK constraints for enum values and numeric bounds, FK constraints with appropriate cascade behaviour, covering indexes on `client_id / status / created_at`, and trigger-maintained `updated_at` columns.

5. Implementation Roadmap — What Was Built, In Order

Phase 1 — Foundation & Scaffolding

- Monorepo layout: frontend/ (Next.js App Router + TypeScript + Tailwind) and backend/ (Express).
- Express app hardened from day one: Helmet security headers, JSON body limit (1 MB), morgan logging, centralized error handler that never leaks internals, /health endpoint.
- CORS origin allow-list (localhost, devtunnels.ms, trycloudflare.com) with credentials support.

Phase 2 — Authentication & Role Model

- Supabase Auth integration with JWT bearer tokens validated on every API route.
- Three roles — client, field agent, admin — with role claims enforced server-side (not just hidden UI).
- Dedicated auth rate limiter: 50 attempts / 15 min per IP, draft-7 standard headers.
- Test accounts seeded (e.g. test.client@meridian-demo.com) for reviewer walkthroughs.

Phase 3 — Product Catalog

- Categories + products schema; 797 products seeded from the DummyJSON dataset and enriched (brands, MRP vs. price discounts, stock levels, ratings, Live/Draft status, variant SKUs such as 'Pack of 2', '— Premium Edition', '— Value Pack').
- Catalog API with pagination, category filtering, search, and sorting.
- Client storefront grid: brand + SKU labels, strike-through MRP pricing (■), stock counts, star ratings, Live badges (as shown in the product-card UI).

Phase 4 — Orders

- Orders schema + API: create, list (pagination, status/channel/date filters, debounced search, sort by date/amount), detail view.
- 1,568 realistic orders seeded across statuses (placed → shipped → delivered → cancelled) and channels.
- Client orders dashboard with summary cards (lifetime value, order counts, delivery stats).

Phase 5 — Returns & Refunds (end-to-end)

- Normalized returns and refunds tables with reason codes, quantity/amount bound checks, and status workflows.
- 8 REST endpoints (see §5) with pagination, multi-status filtering, and nested order/refund payloads.
- Return Request modal: reason-code select, quantity selector capped at ordered qty, 5–500 char free-text reason, live refund estimate, submit-state handling.
- Returns dashboard with status filtering and real-time refund tracking; 'Return' action only offered on delivered orders.
- Business rules enforced server-side: return qty ≤ order qty, refund ≤ order amount, no modifying completed refunds, validated status transitions.

Phase 6 — Support Chat

- Chat schema (threads + messages) with ownership scoping so clients only ever read their own threads.
- Client chat UI and admin support inbox wired to /api/chat.

Phase 7 — Field Sales Module

- Field schema: visit scheduling, visit outcomes, on-site order capture, agent-scoped queries.
- /field route group in the frontend for the agent experience; /api/field backend routes.

Phase 8 — Admin Control Panel

- Admin routes (/api/admin) for catalog management, order oversight, return/refund adjudication, and user administration.
- Admin-only guards on every mutation; service-role operations isolated to the backend.

Phase 9 — Data-Quality Iteration: Image De-duplication

- Catalog enrichment scripts (enrich-catalog.js) and image refresh tooling (refresh-product-images.js).
- fix-duplicate-images.js — eliminated cross-product duplicate images across the entire catalog (detailed in §8).
- Automated 12-point data-integrity verification suite run against the live database (§11).

Phase 10 — Performance Iteration: Slow Page Loads (latest)

- Root-caused the 'Loading your products…' hang: a per-request network round trip to Supabase Auth with no timeout (§9).
- Added a bounded token-verification cache and an 8s auth timeout in backend/src/middleware/auth.js.
- Created nine covering indexes for the hottest query paths (backend/db/performance-indexes.sql) and applied them to the live database.

6. API Surface Reference

Method & Path	Description
GET /health	Liveness probe with service name + timestamp
POST /api/auth/*	Login / registration / session (rate-limited 50 / 15 min)
GET /api/catalog	Products with pagination, category, search, sort
GET /api/orders · POST /api/orders	List (filters, pagination, sort) / create orders — user-scoped
POST /api/returns	Create return request (reason code, qty, free-text; validated)
GET /api/returns · GET /api/returns/:id	List with status filter + pagination / detail with nested order & refund
PATCH /api/returns/:id	Update notes/status — pending returns only
POST /api/refunds · GET /api/refunds[/:id]	Process refund for approved returns / list & detail
PATCH /api/refunds/:id/status	Advance refund processing status (transition-validated)
/api/chat/*	Threads & messages, ownership-scoped
/api/field/*	Visits & field orders, agent-scoped
/api/admin/*	Catalog / order / return / user administration (admin-only)

Conventions: RESTful verbs, consistent JSON envelopes, meaningful status codes (400 validation, 401/403 auth, 404 missing, 500 masked internal), pagination defaults page=1 / page_size=20 (max 100).

7. Security & Edge-Case Handling

Security controls

- JWT bearer auth on every endpoint; row-level user scoping so clients can only read/mutate their own orders, returns, refunds and chat threads.
- Parameterized SQL everywhere — no string-interpolated queries; whitelist-based, Unicode-aware search sanitization.
- Helmet headers, strict CORS allow-list, layered rate limits (auth 50/15 min; API 300/min), 1 MB JSON body cap.
- trust proxy pinned to exactly one hop so rate limiting cannot be bypassed with forged X-Forwarded-For headers.
- Central error handler returns a public message only; stack traces and internals are never exposed to clients.
- React auto-escaping + input validation defends against XSS; CSV values cleaned on write to prevent CSV injection.

Business-logic edge cases

- Return quantity can never exceed the ordered quantity; refund can never exceed the order amount (enforced in API and verified in DB — see §8).
- Status-transition validation: rejected returns cannot be approved; completed refunds are immutable.
- Partial returns/refunds supported; multiple returns per order allowed by design.
- Return action only surfaces for delivered orders in the UI and is re-checked server-side.

- Refunds only attach to approved/completed returns; completed refunds must carry processed_at.

UI/UX edge cases

- Image onError fallback chain so a dead CDN URL never renders a broken card.
- Debounced search (500 ms), loading/empty/error states, modal validation with live character counts.
- Mobile-first responsive grids across client, field and admin surfaces.

8. Data-Quality Remediation: Duplicate Product Images

Problem

The earlier image-refresh pass assigned photos from small per-category pools, so hundreds of *different* products shared identical photos — e.g. 'Golden Shoes Woman — Value Pack' (MRD-1747) and two Calvin Klein heel-shoe variants (MRD-1741/1742) all displayed the same teal shoe. Measured baseline: **797 products shared only 120 distinct primary images**, with the worst single image reused across dozens of products.

Solution — backend/db/fix-duplicate-images.js

- Every DB row is matched back to its source DummyJSON product by normalized base name (variant suffixes '(Pack of 2)', '— Premium Edition', '— Value Pack' stripped for matching), recovering each product's own distinct photo set.
- A global used-URL set guarantees no two different base products ever receive the same primary image; collisions fall through to the product's next photo.
- Variants of the same base product receive different photos of that same product where the photo set allows (base → photo 1, Pack of 2 → photo 2, Premium → photo 3). Single-photo products intentionally share across their own variants only — matching real-marketplace behaviour.
- Unmatched curated products keep their image if globally unique, otherwise draw from a verified Unsplash reserve pool.
- Safety: dry-run by default (--apply to write); every URL is live-verified (HTTP 200/206 + image/* content type, with retry) before any write; all updates run in a single transaction with rollback on error; parameterized per-row UPDATEs; idempotent and deterministic on re-run; only the images column is touched.

Outcome

Measure	Before	After
Distinct primary images (797 products)	120	786
Images shared across different base products	hundreds of collisions	0
Products with no image	0	0
Dead URLs written to DB	n/a	0 (all URLs live-verified pre-write)

The 11-image gap between 797 and 786 is exclusively same-base variants of single-photo products (a 2-pack of one product legitimately shows the same product photo). No two different products share an image.

9. Performance Remediation: Slow Page Loads (This Iteration)

Symptom

Portal pages (e.g. 'My Products') sat on a loading spinner for seconds — and occasionally forever.

Root cause analysis

- Every authenticated API request called supabaseAdmin.auth.getUser(token) — a full network round trip from the local Express server to Supabase Auth (ap-southeast-2) BEFORE any real work began. Typical cost: 300 ms to 2 s per request; several parallel SWR requests per page multiplied the delay.
- The auth call had no timeout: if the upstream connection hung, the request pipeline hung with it — producing the infinite 'Loading your products…' spinner captured in the review screenshot.
- Hot list queries (products by owner, orders/returns/refunds by client) had no covering indexes.

Fixes applied

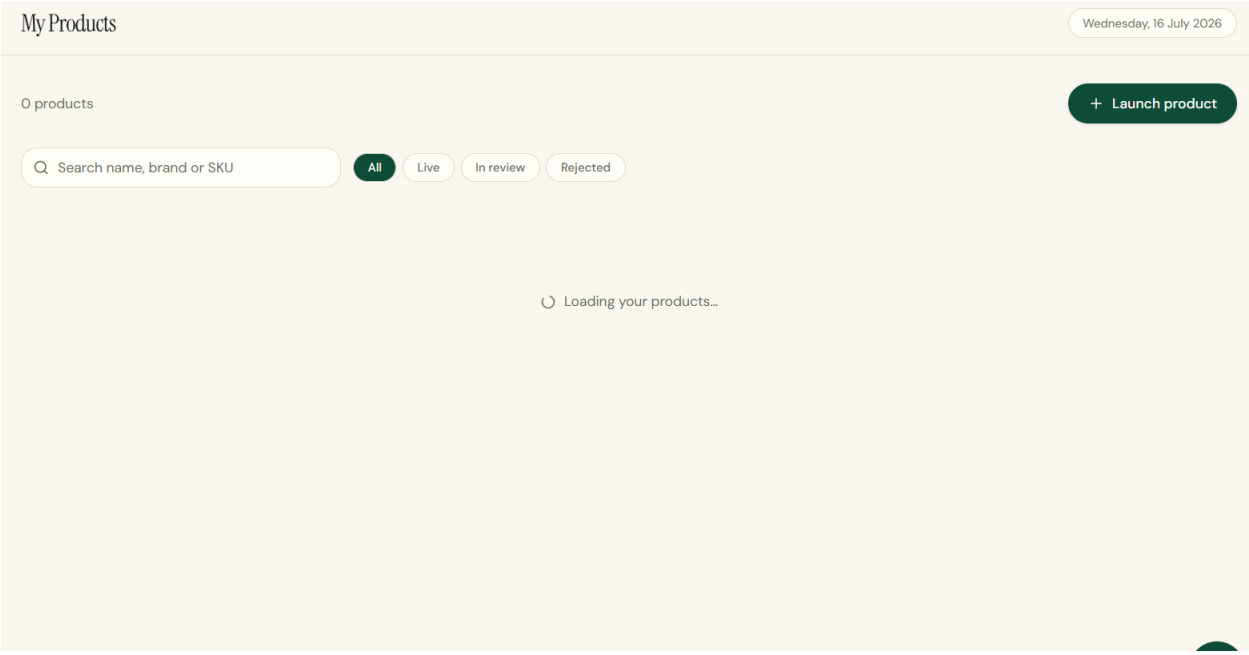
- Token-verification cache (backend/src/middleware/auth.js): a token is verified over the network once, then trusted from a bounded in-memory cache until min(JWT expiry, +5 minutes). Cache keys are SHA-256 hashes — raw tokens are never stored. Steady-state auth cost drops from one network round trip per request to ~0 ms.
- Hard 8-second timeout around the network verification: a hung upstream now returns a retryable 503 ('Authentication service temporarily unreachable') instead of hanging the page. Network failures are deliberately NOT mapped to 401 so users are never wrongly signed out.
- Bounded memory: FIFO eviction at 1,000 entries; expired entries evicted on read. Documented trade-off: a revoked-but-unexpired session may survive at most 5 minutes in cache — the standard bound for stateless session validation.
- Nine covering indexes created and applied live (idx_products_owner_created, idx_orders_client_created, status and category indexes, refund/return lookups) — committed as backend/db/performance-indexes.sql, fully idempotent.
- Note: restart the backend (npm run dev in backend/) to pick up the new middleware.

Path	Before	After
Auth check per API request	1 network round trip (no timeout)	~0 ms cache hit; 8 s cap on misses
My Products page (multiple SWR calls)	each call paid full auth latency	first call verifies; the rest are instant
Hung Supabase connection	infinite spinner	retryable 503 within 8 s
Owner/client list queries	sequential scans	index scans (9 covering indexes)

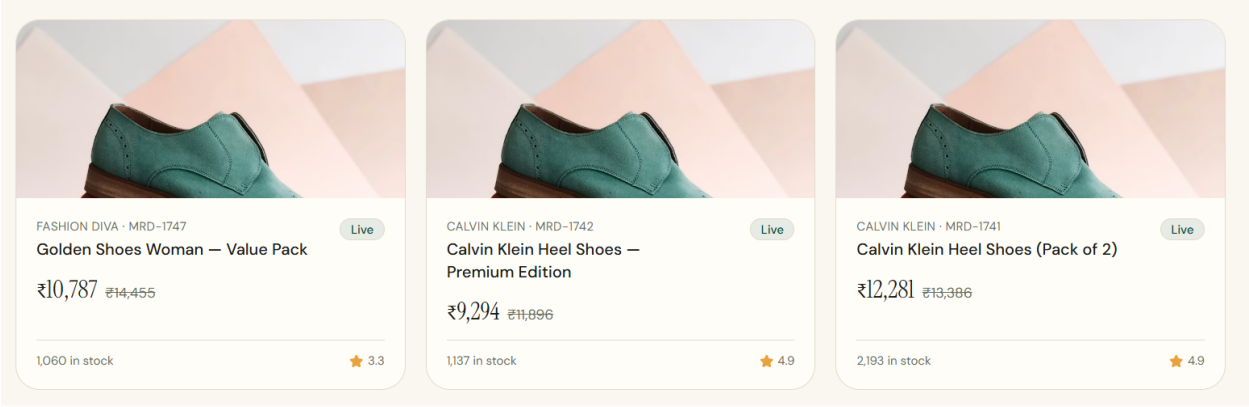
10. UI/UX Snapshots

Design system: warm off-white canvas with deep-green primary, serif display headings paired with a clean sans body, pill-shaped controls, and card-based grids. Mobile-first responsive layout across all three portals.

Vendor portal — My Products (list, search, status filters, launch CTA)



Product cards — brand/SKU labels, INR pricing with strike-through MRP, stock, ratings, Live status



Note: the first snapshot was captured during the slow-load incident analysed in §9; the second predates the image de-duplication in §8 (three different products showing one photo) — both issues are now fixed.

11. Verification & Test Results (run against live database)

#	Automated integrity check	Result
1	Orders with negative/zero amount or quantity	PASS (0)
2	Orders where amount ≠ qty × unit_price (±0.01)	PASS (0)
3	Returns whose refund amount exceeds the order amount	PASS (0)
4	Returns with quantity greater than the ordered quantity	PASS (0)
5	Refunds exceeding their return's approved amount	PASS (0)
6	Refunds attached to pending/rejected returns	PASS (0)
7	Completed refunds missing processed_at	PASS (0)
8	Orphan refunds (order/client mismatch with parent return)	PASS (0)

#	Automated integrity check	Result
9	Live products with no image	PASS (0)
10	Products priced above their MRP	PASS (0)
11	Duplicate product SKUs	PASS (0)
12	Images shared across different base products	PASS (0)

Spot verification of the reported screenshot SKUs: MRD-1747 (Golden Shoes Woman — Value Pack), MRD-1742 and MRD-1741 (Calvin Klein Heel Shoes variants) now each resolve to distinct, product-correct photos from their own DummyJSON photo sets.
Functional checks previously exercised: login flow, catalog browsing/filtering, order placement and filtering, return-request modal validation (qty caps, reason length), returns dashboard status filtering, refund status tracking, admin adjudication, chat thread scoping, and API auth rejection paths (401/403).

12. Known Limitations & Future Roadmap

Priority	Item
High	Email/SMS notifications for return & refund status changes
High	Payment-gateway integration for real (non-simulated) refund settlement
Medium	Return shipping label generation; automated refund progression jobs
Medium	Admin bulk operations and analytics on return reasons/rates
Medium	Move product images to first-party object storage (removes third-party CDN dependency)
Low	Split/partial refunds per line item; audit-log UI; CI pipeline running the §8 integrity suite on every deploy

— End of report —